

现代团队协作架构下的AI驱动智能任务协同看板系统：基于Spec Coding与Trae Agent协作流的研发执行方案

1. 范式转移与执行方案概述

在现代软件工程与团队协作需求日益复杂的背景下，传统的软件开发生命周期正面临深刻的结构性危机。传统模式高度依赖人类在产品经理、前端开发工程师、后端开发工程师以及测试工程师等角色间的明确分工，这种基于人类角色的串行协作往往会导致上下文丢失、沟通成本激增以及技术实现与业务需求之间的严重偏离¹。面对构建“AI驱动的智能任务协同看板”这一高复杂度项目，本报告提出一种颠覆性的研发执行方案：彻底放弃传统的人类角色分工，全面转向基于Trae IDE及其SOLO自治模式的Spec Coding（规范驱动编码）协作流。

这一执行方案的核心在于将繁重的执行层任务委托给由Trae驱动的专业化Agent，而将原本由四人组成的人类团队从代码编写与手动测试的泥潭中解放出来，转变为整个系统架构的编排者、规则的制定者以及质量的终极审核者³。研发协作流被重新构建为由“产品设计Agent”、“研发Agent”与“测试Agent”组成的自动化流水线。依靠Trae内置的Multi-Agent机制，我们为这些Agent配置了极其细粒度的能力模块（Skill），包括需求Skill、原型Skill、表设计Skill、端口设计Skill、OpenSpec规范层系列Skill、编码Skill、用例设计Skill以及自动化测试Skill。

本报告将全面深入地剖析目标系统的技术架构、驱动Agent协作的底层通信协议、Spec Coding的具体实施框架，并详细论述如何利用Trae的IDE模式和SOLO模式，在四人团队的治理下无缝生成涵盖PRD、原型、源码、设计文档、OpenSpec文档、测试用例/脚本/报告、AI辅助日志、PPT及视频等在内的全套标准交付物。这种基于Trae Agentic工作流的重构，不仅是对生产工具的升级，更是对软件研发本体论的重新定义。

2. 目标系统架构：智能任务协同看板的技术解析

在定义Agent协作流之前，必须首先明确目标系统的架构蓝图。目标系统是一个专为现代团队协作设计的AI驱动智能任务协同看板，其核心业务逻辑涵盖用户认证、项目管理、看板拖拽、任务详情、通知提醒以及AI智能推荐等六大核心模块⁵。这些模块的设计不仅需要满足高效的人机交互需求，还需要为后续的AI Agent代码生成提供清晰的技术边界。

系统的前端架构选用Vue 3结合Vue Draggable生态，以构建高响应式的组件化界面。看板系统本质上是一个复杂的状态机，界面的每次拖拽都需要与底层数据状态保持严格的一致性。通过引入Sortable.js的Vue封装（Vue Draggable），系统能够在实现流畅拖拽动画的同时，捕获拖拽事件的生命周期，从而触发向后端发送的状态更新请求⁸。前端应用将被划分为认证视图、项目仪表盘以及核心看板视图等多个层级，确保组件的高复用性与易维护性⁹。

在后端基础设施方面，系统采用Spring Boot微服务架构，并深度集成Spring AI模块以驱动后端的智能化能力¹¹。Spring Boot负责处理核心的业务逻辑封装、通过JWT（JSON Web Tokens）实现的无状态用户认证、基于RBAC（基于角色的访问控制）的项目权限管理，以及通过

WebSocket维持的多客户端实时状态同步¹⁴。关系型数据库（如PostgreSQL）将被用于持久化用户实体、项目元数据以及任务状态轨迹¹⁵。为直观呈现各模块的架构定位与实现机制，下表对系统的核心功能进行了多维度解析。

核心功能模块	技术实现路径与框架支持	功能特性与Agent开发指向
用户认证与授权	Spring Security + JWT，前端Vue Router路由守卫。	提供安全的基础访问控制。研发Agent需通过端口设计Skill实现鉴权拦截器与加密算法集成。
项目与权限管理	关系型数据库结构设计（RBAC模型），RESTful API提供CRUD操作。	支持多租户或多团队空间隔离。表设计Skill需自动生成无冗余的关联表结构与外键约束。
看板拖拽与状态机	Vue 3 + Vue Draggable (Sortable.js)，后端WebSocket实时通信 ⁷ 。	实现平滑UI交互与多端状态同步。要求Agent具备处理并发冲突与操作转换（Operational Transforms）的编码能力 ¹⁴ 。
任务详情与富文本	Vue组件化表单，支持Markdown解析与附件上传。	捕获任务的颗粒度信息。数据库设计需支持JSONB等灵活数据类型以存储扩展字段。
通知提醒系统	消息队列结合WebSocket推送机制，定时任务调度器。	确保任务临近截止或状态变更时实时触达用户。需配置状态监听器与事件驱动架构。
AI智能推荐与解析	Spring AI模块集成大语言模型（如GPT-4o-mini），自然语言处理（NLP）解析器 ¹³ 。	分析用户输入的自然语言（如“明天下午2点完成财务报表”），提取意图、实体与时间戳，自动生成任务并智能推荐优先级与分配人 ⁵ 。

智能任务协同看板的独特价值在于其内嵌的AI智能推荐功能。当用户通过自然语言输入指令时，Spring AI模块会作为中枢，利用大语言模型的自然语言处理（NLP）能力进行意图识别与实体抽取⁵。传统系统依赖表单输入，而该系统则允许用户通过类似“将高优先级的数据库迁移任务分配

给后端团队并设定下周五截止”的对话式命令，由AI直接解析并转化为标准化的任务数据结构，随后通过函数调用（Function Calling）在看板上自动创建相应的任务卡片并匹配推荐的执行者¹⁷。这种智能化的交互模式极大地降低了团队管理的认知负荷，提升了协作流转的效率。

3. 代理编排基石：模型上下文协议（MCP）与函数调用机制

为了让产品设计Agent、研发Agent和测试Agent能够真实地执行架构设计、代码编写与软件测试，它们必须具备与外部物理世界和软件系统交互的能力。单纯依靠对话式的语言模型无法完成复杂的工程部署，因此，本执行方案的底层架构深度依赖于模型上下文协议（Model Context Protocol, MCP）与大语言模型的函数调用（Function Calling）机制。

3.1. 模型上下文协议（MCP）的架构整合

在传统的AI应用开发中，使得语言模型访问外部数据或工具面临着严重的“N×M问题”：每一个新的AI模型与每一种外部工具之间都需要定制化的集成，导致架构高度脆弱且难以维护²¹。MCP作为一种开源的通用连接标准，从根本上解决了这一问题。它被设计为“AI的USB-C接口”，提供了一种标准化的Client-Server架构，通过JSON-RPC协议在AI模型与外部数据源之间建立安全的双向通信¹⁵。

在本项目中，MCP主机（即Trae的底层编排器）将管理与多个MCP服务器的连接。产品设计Agent将通过Figma MCP服务器直接读取UI原型库的设计变量；研发Agent将通过GitHub MCP服务器管理代码分支、拉取请求，并使用本地文件系统MCP服务器直接读取、修改和写入Vue与Spring Boot的源码文件¹⁵；测试Agent则利用Playwright MCP服务器控制无头浏览器执行UI自动化测试，甚至通过PostgreSQL MCP服务器在测试前后执行数据清理脚本¹⁵。这种架构确保了Agent并非在真空中“幻觉”出代码，而是基于系统的真实当前状态（Context）进行确定性的操作推理²⁴。

3.2. 函数调用（Function Calling）的执行逻辑

MCP解决了连接问题，而函数调用（亦称工具调用）则赋予了Agent决定“何时”以及“如何”使用这些连接的自主决策能力²⁶。当大型语言模型（如GPT-4或Claude 3.5）接收到执行任务的指令时，系统会同时提供一个定义了可用工具及其参数格式的JSON Schema²⁶。

例如，当研发Agent被要求“根据设计文档创建用户表”时，它不会仅仅输出一段SQL文本。相反，语言模型会评估当前语境，决定调用名为execute_sql_migration的工具，并严格按照Schema输出包含完整建表语句和外键约束的结构化JSON参数²⁶。编排层捕获该请求，通过对应的MCP服务器在真实数据库中执行该操作，并将执行成功或失败的反馈（包括错误堆栈）返回给语言模型，从而实现闭环的自愈修复过程²⁸。这一机制构成了Agent驱动模式的核心控制流，确保了机器动作的精确性与可追溯性。

4. 规范驱动开发（SDD）：OpenSpec框架与EARS语法体系

过去几年中，基于“Vibe Coding”（凭感觉编码）的AI辅助开发模式暴露出巨大缺陷。在缺乏严格规范的情况下，AI往往会在长对话中丢失上下文，将简单的UI修改演变为破坏性的全局重构，导致系统架构迅速失控³⁰。为杜绝此类风险，本方案强制采用规范驱动开发（Spec-Driven Development, SDD）范式，并利用OpenSpec框架结合Trae CLI作为整个研发管线的唯一事实来源与执行标准。

4.1. OpenSpec核心制品生命周期

OpenSpec框架彻底将“需求规范的定义”与“代码更改的执行”解耦³³。任何新特性的开发或修改都必须经过明确的提议、设计、实现与归档阶段。在此过程中，产品设计Agent在OpenSpec规范层系列Skill的约束下，必须依次产出以下标准化的Markdown制品。

OpenSpec核心制品文件	所在生命周期阶段	职责定义与系统影响解析
proposal.md	变更提议（Propose）	阐明变更动机。回答“为什么要实施这项更改？”以及“预期解决什么问题？”。同时界定更改的影响范围（如涉及的API、数据库依赖与前端视图） ³⁴ 。
specs/*.md (Delta Specs)	需求规范（Specify）	采用增量式规范记录系统的行为变化。不重复已有需求，而是通过ADDED、MODIFIED、REMOVED精确描述增量逻辑，充当测试Agent验证的基准 ³¹ 。
design.md	技术架构（Design）	规定技术实施路径。记录表结构变化、RESTful接口设计以及架构决策（如使用哪种状态机模式处理看板拖拽逻辑），约束研发Agent的代码生成边界 ³¹ 。
tasks.md	实施清单（Tasking）	包含5至15个颗粒度极小的实现步骤检查表。步骤需具有独立可测试性并按依赖关系排序（如先建库后写接口）。研发Agent严格按照此清单逐项执行 ³¹ 。

4.2. EARS需求语法的强制约束

为了让specs/*.md中的自然语言需求既能被人类团队轻松审核，又能被测试Agent无歧义地解析以生成测试用例，产品设计Agent被严格要求使用“简化需求语法框架”（Easy Approach to Requirements Syntax, EARS）³⁹。EARS通过一套极简的规则集和逻辑触发词，有效消除了自然语言中固有的模糊性³⁹。通过模式匹配，所有系统行为都被强制划分为以下几种场景，下表展示了EARS语法在智能任务看板项目中的具体应用。

EARS语法模式	逻辑结构约束	智能任务协同看板项目应用示例
无处不在的需求 (Ubiquitous)	The <system> shall <response>	智能任务协同看板系统必须（shall）提供毫秒级的WebSocket状态同步能力。 ⁴²
事件驱动的需求 (Event-driven)	When <trigger>, the <system> shall <response>	当（When）用户将任务卡片拖拽至不同的看板列时，系统必须（shall）将任务状态更新为与目标列相匹配的状态。 ⁴²
状态驱动的需求 (State-driven)	While <state>, the <system> shall <response>	当（While）应用程序处于离线状态时，系统必须（shall）将针对任务的更新暂存于浏览器的本地存储（Local Storage）中。 ³⁹
无用/异常行为需求 (Unwanted)	If <trigger>, then the <system> shall <response>	如果（If）在状态同步过程中发生版本冲突，那么系统必须（shall）向用户弹出一个冲突解决对话框。 ⁴⁰
复杂逻辑需求 (Complex)	While <state>, When <trigger>, the <system> shall <response>	当（While）用户处于仅具有只读权限的项目空间时，当（When）用户尝试拖拽任务卡片，系统必须（shall）拒绝该操作并触发访问受限的通知提醒。 ³⁹

4.3. Trae驱动的研发闭环控制

OpenSpec工作流不依赖于自由格式的聊天窗口，而是通过类似于Git的Trae CLI（trae-cli）或

Trae内置指令驱动系统的状态流转。首先，在Trae中执行/opsx:propose命令触发变更提议阶段，产品设计Agent生成上述全套设计制品。紧接着，系统运行自动化校验脚本对目录结构和EARS语法进行验证³⁴。随后，执行/opsx:apply命令进入应用阶段。此时，可以一键切换至Trae的SOLO模式，研发Agent在SOLO模式下完全接管控制权，读取技术决策与任务清单，利用其编码Skill通过MCP对本地源码进行实质性的编写与修改，并在完成每一步后主动在tasks.md中标记完成（- [x]）以追踪进度。最后，在测试通过后执行/opsx:archive命令，AI将增量规范合并至主线文档，并将此次变更的执行记录归档以备审计验证³⁷。这种控制流彻底消除了AI的自由发散性，使其行为完全可控。

5. Trae Agent拓扑结构与核心Skill库配置

在此AI Agent驱动模式中，我们依托Trae平台的Multi-Agent编排能力，构建了由“产品设计Agent”、“研发Agent”以及“测试Agent”组成的三元拓扑网络。Trae支持在IDE交互模式和SOLO自主模式之间无缝切换，赋予了各Agent专业身份与操作边界的机制被称为“Skill”——一系列内嵌了上下文约束、格式准则以及可用MCP工具声明的配置文件。以下深入解析各Agent在Trae中的职责及其配置的核心Skill体系。

5.1. 产品设计Agent（Product Design Agent）

产品设计Agent位于价值链的最前端，主要在Trae的IDE模式或Chat/Builder模式下与人类进行高频交互，负责将模糊的商业愿景和用户需求转化为结构化的技术规格。

- **需求Skill (RequirementSkill):** 这一核心能力不仅赋予了Agent理解现代协作业务场景的能力，更强制其在梳理用户认证、项目管理等需求时必须采用EARS格式编写。它使得Agent能够主动追问需求边界，穷举异常分支，并在编写文档前理清系统逻辑的因果关系³⁴。
- **原型Skill (PrototypeSkill):** 借助MCP与UI设计工具（如Figma）的集成通道，该Skill使得Agent能够根据需求解析结果，以编程方式生成或操作界面元素的布局设计。Agent可基于Tailwind CSS规范或Vue组件库标准，直接生成看板界面的视觉隐喻与交互层级约束，形成可交付的UI原型图²³。
- **OpenSpec规范层系列Skill (OpenSpec Specification Skills):** 这一系列能力限定了产品设计Agent的文件输出格式。它规定Agent必须生成包含proposal.md和tasks.md等在内的标准工件，必须使用URL安全的变更ID建立正确的目录结构，并且禁止在提议阶段生成任何可执行的源代码³⁴。

5.2. 研发Agent（R&D Agent）

研发Agent是系统的建设者。为了追求极致效率，该Agent主要在Trae的SOLO模式下运行，负责阅读设计蓝图，并将其自主翻译为能在Spring Boot和Vue 3环境下高效运行的源代码体系。

- **表设计Skill (TableDesignSkill):** 专门应对数据持久化层的设计挑战。当识别到“任务详情”或“用户认证”需求时，该Skill驱动Agent综合考虑索引优化、外键约束、JSONB扩展字段等高级特性，自动生成符合第三范式的高效PostgreSQL数据库建表脚本或Spring Data JPA的Entity映射模型³⁴。
- **端口设计Skill (PortDesignSkill):** 该Skill规范了系统内外的数据交换契约。它要求研发Agent在设计API时必须遵循RESTful规范，定义清晰的请求体与响应体结构，提供标准的

HTTP状态码错误处理策略，并在处理实时看板拖拽等事件时设计严密的WebSocket通信载荷协议⁴⁷。

- **编码Skill (CodingSkill):** 作为研发Agent的引擎核心，编码能力受到Trae项目规则的严格约束。该文件指明了技术栈（Spring Boot 3, Vue 3, Vue Draggable）及设计模式要求。编码Skill确保Trae SOLO能够安全地调用本地文件系统MCP工具，分层编写控制器、服务层与持久层代码，并强制要求其严格按照tasks.md中的步骤顺序单步执行，避免上下文跨越导致的灾难性重构。

5.3. 测试Agent（Testing Agent）

测试Agent扮演着质量守门员的角色，同样在Trae SOLO模式下被激活，作为研发Agent的对抗性实体存在，确保所有的代码产出完全符合EARS规范中的定义。

- **用例设计Skill (TestCaseDesignSkill):** 该Skill赋予Agent从specs/*.md文件中逆向解析出测试矩阵的能力。它能基于边界值分析和等价类划分原则，将EARS格式的自然语言需求转换为结构化的测试用例列表，不仅包含正向路径（Happy Path），更着重于覆盖各种异常行为（Unwanted Behaviors）的负向测试³⁴。
- **自动化测试Skill (AutomationTestSkill):** 这一能力使得测试Agent能够直接编写并执行可运行的测试脚本。针对后端逻辑，它能生成JUnit和Mockito测试代码；针对前端看板的拖拽交互，它利用MCP调度Playwright无头浏览器执行端到端（E2E）的UI自动化测试⁴⁸。此外，当测试失败时，该Skill支持执行根本原因追踪（root-cause-tracing），将带有精确错误堆栈信息的日志反馈给研发Agent，触发代码自愈循环⁴⁸。

6. 四人协作流重塑：面向Trae架构的人类角色与治理矩阵

在放弃传统的“PM-前端-后端-测试”人类角色划分后，这并不意味着人类的退出，而是意味着劳动形式向更高维度的升级。四人团队的职责发生了根本性转变，从微观层面的任务执行者转变为宏观层面的系统架构师与Trae Agent生态的治理者²。团队成员的信条变为“你在视觉上保持控制，让AI处理复杂性”（You stay visually in control while AI handles the complexity）。以下是重构后的四人制人类团队角色与责任分配（RACI）矩阵。

重构后的人类角色	核心治理职责与操作范围	与Trae生态的交互模式	在OpenSpec生命周期中的定位
系统架构师 / Agent 设计师 (Systems Architect / Agent Designer)	负责顶层设计，定义Agent的权责边界、MCP工具访问权、升级阻断逻辑，并编撰整个项目的技术宪法文件（project.md / constitution.md） ⁴	在Trae中编写并调优各Agent的SKILL.md配置，管理Agent大模型的提示词上下文，防止系统性架构漂移。	处于项目初始化阶段；负责技术设计方案（design.md）的最终拍板与校验。

	。		
集成 / 粘合工程师 (Integration / Glue Engineer)	负责数字基础设施的管理。处理API鉴权密钥、配置CI/CD自动化流水线、搭建数据库实例，并解决MCP服务与外部系统集成的底层网络与权限问题 ⁴ 。	部署并维护MCP服务器集群。利用trae-cli确保研发Agent与测试Agent拥有安全、稳定且畅通无阻的通道访问本地仓库及沙箱环境。	贯穿整个实施与交付阶段；保障环境的一致性。
评估 / 审计工程师 (Eval Engineer)	专注于系统反馈循环。负责质量护栏的建设，执行红蓝对抗测试，监控模型推理的准确性，并建立长期的数据漂移与性能退化检测机制 ⁴ 。	审计测试Agent生成的报告。通过Trae内置的AI调试工具和辅助日志追踪异常行为，识别Agent在推理过程中产生的逻辑幻觉。	主导测试校验与变更归档阶段；输出风险评估报告。
领域专家 / 业务仲裁者 (Domain Expert / Human-in-the-Loop)	代表最终用户与商业利益。提供业务背景，解决业务逻辑冲突，是衡量系统交付质量与需求贴合度的最终仲裁机构 ⁴ 。	在Trae的IDE交互模式下发起高阶指令。审阅产品设计Agent产出的全套需求规范（proposal.md），充当决策链条中的“人工在环”节点。	主导提议阶段；掌控项目实施的推进开关与最终成果验收。

通过这种矩阵式治理结构，四人团队能够以指数级的效率杠杆指挥由Trae Agent构成的“数字开发军团”。人类无需再逐行排查Vue组件的渲染错误或调试Spring Boot的依赖注入失败，而是将精力集中于系统安全性、架构演进以及商业逻辑的正确性上³。

7. 全生命周期研发执行路径

基于前述的架构解析、协议支撑与治理模型，我们将实际构建智能任务协同看板项目的生命周期拆解为高度纪律化的、由Trae链式调用的全自动化流转路径。整个研发管线严格遵循OpenSpec的方法论，并在Trae的IDE与SOLO双模式间自如切换。

首先，项目进入**初始化与宪法确立阶段**。集成工程师通过Trae集成终端执行openspec init命令，系统自动生成包含specs/（长期规范库）、changes/（变更提议区）以及archive/（历史归档区）的核心目录结构³³。与此同时，系统架构师在根目录植入技术宪法文件，确立包括前端Vue 3、后端Spring Boot 3、统一禁止使用本地缓存而强制依赖Redis等架构级硬约束³³。接着，所有的MCP服务器实例启动，Trae的三个Agent加载其对应的Skill模块准备就绪。

项目推进至**提议与蓝图设计阶段**。领域专家在Trae的IDE交互模式下输入高阶自然语言指令，例

如：“开发包含AI推荐功能的任务详情与通知提醒模块”。系统触发/opsx:propose操作。产品设计Agent立刻响应，激活需求Skill，利用MCP搜集项目现有上下文，迅速在变更目录下生成包含“为什么要加入AI推荐”以及“通知提醒的影响范围”的proposal.md³⁴。接着，Agent利用原型Skill对接Figma，生成任务详情侧边栏的界面原型结构²³。关键在于，Agent将采用EARS语法编排specs/文档，制定形如“当任务状态由研发Agent变更为完成时，系统必须向创建者推送实时通知”的无歧义规则⁴²。领域专家对生成的蓝图进行审查，确认无误后通过阻断门，授权进入技术设计流程⁴⁴。

此时由产品设计Agent继续输出**技术架构与实施清单**。其自动生成design.md，使用表设计Skill规划通知日志表与任务历史表，使用端口设计Skill定义RESTful接口文档及通知模块的WebSocket协议格式³⁴。进而，极其重要的tasks.md被生成，该文件将庞大的需求拆解为十几个包含复选框的原子实现步骤（如：“1. 创建Notification Entity；2. 编写Notification Service；3. 集成Vue组件触发逻辑”）³¹。系统架构师最终在IDE中审核设计文档，验证表结构设计的合理性。

设计确认后，管线正式进入**自动化代码执行阶段**。系统执行/opsx:apply命令，并一键切换至Trae SOLO模式，唤醒完全自治的研发Agent。研发Agent受编码Skill约束，以tasks.md为唯一行动指南，开始利用本地文件操作MCP进行编码。它从第一条任务开始，编写Spring Boot控制器、构建Vue Draggable看板的拖拽事件监听器，并逐步完成数据持久化层与前端组件的对接⁷。每完成一个步骤，研发Agent主动在tasks.md中标记打钩完成（- [x]）⁴⁴。这种透明的进度同步让集成工程师能够实时监控代码库的演进，防止模型陷入无休止的生成死循环。

代码编写完毕后，无缝切入**基于Agent对抗的自动化测试与修复阶段**。测试Agent在Trae SOLO模式下被激活，其利用用例设计Skill直接读取产品设计Agent输出的EARS规范文档，理解原本期望的系统行为。随即，测试Agent通过自动化测试Skill生成对应的测试代码。针对看板的拖拽功能，测试Agent调用Playwright MCP，模拟真实用户的光标移动，验证任务卡片拖拽后的状态更新是否同步到了后端⁴⁸。若测试未能通过，测试Agent会将详细的错误快照和异常日志发回给研发Agent。研发Agent根据追踪结果定位并修改代码，重新提交。这一“红蓝对抗”循环将在Trae底座上持续进行，直至所有测试用例全数通过，Eval工程师从中抽取测试报告进行最终审计⁴。

最后，项目流转至**合规归档与交付生成阶段**。当所有的代码合入主干且测试绿灯通过，执行/opsx:archive指令。此次开发任务产生的Delta Specs被正式合并成为项目长期的系统规范库，变更目录被移入审计归档区³⁷。至此，整个全自动化研发闭环完成，所有交付物已在流程中被同步孕育而生。

8. 交付物生成机制与AI辅助日志标准化

在放弃传统人力分工的项目交付要求中，如何确保原本依赖人工撰写的复杂交付物（如PPT、视频、详尽日志等）得以高质量输出，是验证Agent模式成功与否的关键。本方案通过管线内嵌的机制，将所有的日常研发活动自动聚合转化为了符合审计要求的标准交付物集。下表详细说明了各项要求交付物的生成节点与机制。

核心交付物类别	负责主体 / 生成工具	生成阶段与技术机制解析
---------	-------------	-------------

PRD与设计文档 (PRD & Design Docs)	产品设计Agent	在提议（Propose）与设计（Design）阶段通过OpenSpec规范层自然产出。包含了由EARS语法构成的业务规格集以及design.md中的系统架构与API定义 ³¹ 。
UI原型 (Prototypes)	产品设计Agent / Figma MCP	在提议阶段利用原型Skill结合UI代码生成逻辑或Figma API自动化渲染生成，确立看板视图与任务详情的组件形态 ²³ 。
源码 (Source Code)	研发Agent / Trae SOLO	核心应用阶段（Apply），根据tasks.md的序列指示，利用编码Skill在SOLO模式下编写并直接提交Vue与Spring Boot工程文件。
测试用例/脚本/报告 (Test Artifacts)	测试Agent / Playwright MCP	在测试阶段通过用例设计Skill逆向解析EARS生成逻辑矩阵；通过自动化测试Skill生成并执行JUnit与Playwright脚本，捕获结果输出结构化的测试审计报告 ³⁴ 。
AI辅助日志 (AI Logs)	Trae编排控制台监控器	全局实时捕获。以结构化格式记录所有AI交互，用于诊断模型的推理链路 with 工具调用历史。
PPT与功能演示视频 (PPT & Video)	产品设计Agent / 测试Agent	归档后（Archive），产品设计Agent从历史文档提取摘要生成结构化Markdown格式的幻灯片纲要；测试Agent利用Playwright测试过程中的录屏功能捕获拖拽与AI命令功能的UI交互视频片段 ⁴⁸ 。

在这批交付物中，AI辅助日志（AI Auxiliary Logs）的标准化与深度追踪能力尤为关键。在Trae高度自治的SOLO系统中，如果智能任务协同看板在运行时出现了任务状态更新丢失的Bug，人类开发人员无法像过去那样仅通过查看代码提交历史来理解背后的业务意图。必须追溯至语言模型做出该代码修改决策时的瞬时思维状态。

为实现这一目标，执行方案要求所有AI辅助日志必须遵循企业级的标准化日志规范（如Syslog RFC 5424协议或JSON序列化格式），以便能够被现有的基础设施监控平台（如Grafana Loki、Splunk或AWS CloudWatch）直接读取与分析⁵³。日志体系不仅记录常规的警告和错误，更将每次大模型推理的过程解构为强制性的DID / INSIGHT / NEXT-STEP（做了什么 / 认知洞察 / 下一步行动）代码块结构³²。具体而言，日志将精确捕获每一次交互的chat_id、被触发的Skill名称、具体调用的MCP工具函数（如read_file、execute_sql_migration）、此次推理消耗的Token数量，以及促使Agent做出相关修改的自然语言解释³²。Eval工程师可以根据这一标准化追踪日志，轻易地重构成“研发Agent在读取了何种规范后，调用了哪个数据库文件，最终导致了哪个Vue组件的重写”的完整因果链条，从而保障了系统的绝对可控与可审计性²⁴。

9. 结论

将传统的软件研发组织结构解构，并全面重塑为以Spec Coding为核心的AI Agent驱动模式，标志着软件工程从劳动密集型产业向高度自动化智力密集型产业的历史性跨越。针对包含用户认证、项目流转、看板状态机、通知系统及智能化NLP解析在内的复杂智能任务协同看板项目，本方案验证了抛弃“产品-前端-后端-测试”陈旧人力划分范式的可行性与巨大优越性¹。

通过构建依托Trae平台的“产品设计-研发-测试”三元AI Agent协作流，并辅以高度专业化的八大核心Skill集，系统实现了从抽象商业需求到可用工程代码的端到端自动化编译。其中，模型上下文协议（MCP）与函数调用技术打破了AI应用的信息孤岛，赋予了语言模型操纵文件、操作数据库及控制浏览器的物理抓手¹⁵；而基于OpenSpec框架与EARS语法的规范驱动开发（SDD），则在AI不可控的“发散思维”外层建立起了一道坚不可摧的逻辑围栏，确保了变更提议、技术设计到按单执行步骤的绝对确定性与高可控性³³。

更为重要的是，借助于Trae独有的无缝双模切换能力——在IDE交互模式下进行强有力的人工宏观把控，在SOLO模式下赋予AI最大化的底层自治执行权——被解放出来的四名人类团队成员成功转型为拥有更高系统杠杆率的架构师、集成者、评估者与业务领域仲裁者，建立起严密的现代Agent生态治理矩阵。这种新范式不仅高保真地交付了全部指定的系统源码与运行环境，更通过流水线副产品的方式，自动化、标准化地输出了涵盖PRD、架构设计文档、高质量UI原型、多维度自动化测试报告、标准化AI追踪日志，乃至全功能演示视频与PPT等全维度的工程资产体系³¹。本报告所确立的研发执行方案，不仅为智能看板项目提供了最高效率的落地路径，更为未来大规模推广基于Trae的企业级Agentic自主研发 workflows 提供了体系化、可复制的实施蓝本。

Works cited

1. AI PRD Generator: Create Product Requirements Fast - Miro, accessed May 16, 2026, <https://miro.com/ai/product-development/ai-prd/>
2. Agentic AI: The End of Roles as You Know Them | by David Williams | Medium, accessed May 16, 2026,

- <https://medium.com/@dpwilliams03/agentic-ai-the-end-of-roles-as-you-know-them-c2ab2af3b4d0>
3. Agentic AI Engineering Team Playbook: Coding, Review, Ops 2026 - Digital Applied, accessed May 16, 2026, <https://www.digitalapplied.com/blog/agentic-ai-engineering-team-playbook-coding-review-ops-2026>
 4. The future team composition in agentic development? : r/AI_Agents - Reddit, accessed May 16, 2026, https://www.reddit.com/r/AI_Agents/comments/1s4p39f/the_future_team_composition_in_agentic_development/
 5. 5 Best AI Task Management Tools in 2026 (Tested) - Morgen, accessed May 16, 2026, <https://www.morgen.so/blog-posts/ai-task-manager>
 6. VS Code Agent Kanban: Task Management for the AI-Assisted Developer : r/vscode - Reddit, accessed May 16, 2026, https://www.reddit.com/r/vscode/comments/1rownyy/vs_code_agent_kanban_task_management_for_the/
 7. Vue.js Kanban Board: The Development Process - Auth0, accessed May 16, 2026, <https://auth0.com/blog/vuejs-kanban-board-the-development-process/>
 8. Draggable Kanban Board - Binrcode, accessed May 16, 2026, <https://www.binrcode.com/resources/draggable-kanban-board>
 9. Creating a Kanban Board with Vue Draggable | Blog - Jscrambler, accessed May 16, 2026, <https://jscrambler.com/blog/kanban-board-vue-draggable>
 10. Building a Simple Kanban Board with Drag and Drop in Vue 3 and Tailwind CSS — Part 1, accessed May 16, 2026, <https://medium.com/@hojny.adrian/building-a-simple-kanban-board-with-drag-and-drop-in-vue-3-and-tailwind-css-part-1-7ecf92b9ce53>
 11. Build a Full-Stack AI Agent in Java | Spring Boot & Spring AI - YouTube, accessed May 16, 2026, <https://www.youtube.com/watch?v=x0MfOcz-SV8>
 12. Building Effective Agents with Spring AI (Part 1), accessed May 16, 2026, <https://spring.io/blog/2025/01/21/spring-ai-agentic-patterns/>
 13. How AI Can Be Integrated Into a Spring Boot Applications - Medium, accessed May 16, 2026, <https://medium.com/@swatisv11/how-ai-can-be-integrated-into-a-spring-boot-applications-spring-ai-779f88438a65>
 14. ChatPRD - The #1 AI Platform for Product Managers, accessed May 16, 2026, <https://www.chatprd.ai/>
 15. Introducing the Model Context Protocol - Anthropic, accessed May 16, 2026, <https://www.anthropic.com/news/model-context-protocol>
 16. Building a Kanban Board with Vuejs and Vue Draggable in 15 Minutes - Diligent Dev, accessed May 16, 2026, <https://www.youtube.com/watch?v=E8f5JR-K5DI>
 17. Implementing an AI Assistant with Spring AI - Baeldung, accessed May 16, 2026, <https://www.baeldung.com/spring-ai-assistant>
 18. What Is NLP (Natural Language Processing)? - IBM, accessed May 16, 2026, <https://www.ibm.com/think/topics/natural-language-processing>
 19. What is Natural Language Processing? - NLP Explained - AWS, accessed May

- 16, 2026, <https://aws.amazon.com/what-is/nlp/>
20. Building a Scheduling Assistants with SpringAI | GridDB: Open Source Time Series Database for IoT, accessed May 16, 2026, <https://www.griddb.net/en/blog/scheduling-assistants-springai/>
21. What is MCP (Model Context Protocol)? | Data Science Collective, accessed May 16, 2026, <https://medium.com/data-science-collective/what-is-mcp-bbea288586a3>
22. Architecture overview - What is the Model Context Protocol (MCP)?, accessed May 16, 2026, <https://modelcontextprotocol.io/docs/learn/architecture>
23. 30+ MCP Ideas with Complete Source Code - DEV Community, accessed May 16, 2026, <https://dev.to/copilotkit/30-mcp-ideas-with-complete-source-code-d8e>
24. PaulDuvall/ai-development-patterns: A comprehensive collection of AI development patterns for building software with AI assistance, organized by implementation maturity and development lifecycle phases. Includes Foundation, Development, and Operations patterns with practical examples and anti-patterns. · GitHub, accessed May 16, 2026, <https://github.com/paulDuvall/ai-development-patterns>
25. MCP is Dead - Why no one uses Model Context Protocol!, accessed May 16, 2026, <https://www.youtube.com/watch?v=sPtTDDXsew0>
26. Function Calling with LLMs - Prompt Engineering Guide, accessed May 16, 2026, https://www.promptingguide.ai/applications/function_calling
27. Function calling | OpenAI API, accessed May 16, 2026, <https://developers.openai.com/api/docs/guides/function-calling>
28. Conversation Routines: A Prompt Engineering Framework for Task-Oriented Dialog Systems, accessed May 16, 2026, <https://arxiv.org/html/2501.11613v3>
29. User-Aligned Functions to Improve LLM-to-API Function-Calling Accuracy - Medium, accessed May 16, 2026, <https://medium.com/@patc888/user-aligned-functions-to-improve-llm-function-calling-accuracy-to-apis-2192fde6ce67>
30. From Vibe Coding to Spec-Driven Development - Towards Data Science, accessed May 16, 2026, <https://towardsdatascience.com/from-vibe-coding-to-spec-driven-development/>
31. Stop Vibe Coding. Start Building with OpenSpec. | by Abhinav Dobhal | Medium, accessed May 16, 2026, <https://medium.com/@abhinav.dobhal/stop-vibe-coding-start-building-with-openspec-b713cc6bb475>
32. Spec-driven agentic coding is quietly making us worse at the job of supervising agents, accessed May 16, 2026, https://www.reddit.com/r/AI_Agents/comments/1ta2dfz/specdriven_agentic_coding_is_quietly_making_us/
33. A Practical Development Guide Based on OpenSpec + Claude CLI | by HBLOG - Medium, accessed May 16, 2026, <https://jxausea.medium.com/a-practical-development-guide-based-on-openspec-cli-aude-cli-26da7df71356>
34. openspec-proposal-creation | Skills ... · LobeHub, accessed May 16, 2026, <https://lobehub.com/de/skills/forztf-open-skilled-sdd-openspec-proposal-creation>

35. OpenSpec: Make AI Coding Assistants Follow a Spec, Not Just Guess, accessed May 16, 2026, <https://recca0120.github.io/en/2026/03/08/openspec-sdd/>
36. OpenSpec/docs/customization.md at main - GitHub, accessed May 16, 2026, <https://github.com/Fission-AI/OpenSpec/blob/main/docs/customization.md>
37. OpenSpec/docs/getting-started.md at main - GitHub, accessed May 16, 2026, <https://github.com/Fission-AI/OpenSpec/blob/main/docs/getting-started.md>
38. openspec-proposal-creation - Skill - Smithery, accessed May 16, 2026, <https://smithery.ai/skills/forztf/openspec-proposal-creation>
39. Adopting the EARS Notation to Improve Requirements Engineering - Jama Software, accessed May 16, 2026, <https://www.jamasoftware.com/requirements-management-guide/writing-requirements/adopting-the-ears-notation-to-improve-requirements-engineering/>
40. Alistair Mavin EARS: Easy Approach to Requirements Syntax | Official Guide, accessed May 16, 2026, <https://alistairmavin.com/ears/>
41. Analyze Your Requirements Against the Easy Approach to Requirements Syntax (EARS) Using Inflectra.ai, accessed May 16, 2026, <https://www.inflectra.com/Company/Article/analyze-your-requirements-ears-using-inflectra-ai-1916.aspx>
42. EARS: The Easy Approach to Requirements Syntax | by Oguz Senna | ParamTech | Medium, accessed May 16, 2026, <https://medium.com/paramtech/ears-the-easy-approach-to-requirements-syntax-b09597aae31d>
43. Feature Request: EARS (Easy Approach to Requirements Syntax) Integration · Issue #1356 · github/spec-kit, accessed May 16, 2026, <https://github.com/github/spec-kit/issues/1356>
44. How to make AI follow your instructions more for free (OpenSpec) - DEV Community, accessed May 16, 2026, <https://dev.to/webdeveloperhyper/how-to-make-ai-follow-your-instructions-more-for-free-openspec-2c85>
45. OpenSpec - Lightweight & portable spec driven framework for AI coding assistants!, accessed May 16, 2026, <https://forum.cursor.com/t/openspec-lightweight-portable-spec-driven-framework-for-ai-coding-assistants/134052>
46. How to Use AI in Software Development: 7 Best Practices & Examples for Engineering Teams - Jellyfish, accessed May 16, 2026, <https://jellyfish.co/library/ai-in-software-development/use-cases-and-best-practices/>
47. OpenSpec Custom Schemas | intent-driven.dev, accessed May 16, 2026, <https://intent-driven.dev/blog/2026/02/12/openspec-custom-schemas/>
48. ComposioHQ/awesome-claude-skills - GitHub, accessed May 16, 2026, <https://github.com/ComposioHQ/awesome-claude-skills>
49. Diving Into Spec-Driven Development With GitHub Spec Kit - Microsoft for Developers, accessed May 16, 2026, <https://developer.microsoft.com/blog/spec-driven-development-spec-kit>
50. AI Agent Orchestration: Your Guide On How to Make Agents Work Together |

- *instinctools, accessed May 16, 2026,
<https://www.instinctools.com/blog/multi-agent-orchestration/>
51. Agents, human agency, and the opportunity for organizations - Microsoft, accessed May 16, 2026,
<https://www.microsoft.com/en-us/worklab/work-trend-index/agents-human-agency-and-the-opportunity-for-every-organization>
 52. openspec-workflow | Skills Marketplace - LobeHub, accessed May 16, 2026,
<https://lobehub.com/skills/arithmetic-copilot-plugins-openspec-workflow>
 53. Software Devs Picked These Log Formats | by Mike Vincent - Medium, accessed May 16, 2026,
<https://mike-vincent.medium.com/software-devs-picked-these-2-log-formats-f6bcd2ef7123>
 54. Blog Post - AI-Assisted Greenfield Software Development, Part 2: Core Instructions, accessed May 16, 2026,
<https://www.codemag.com/Blog/AIPractitioner/AIAGSD2>
 55. I Built an AI Log Analyzer Using MCP — Here's How It Works | by Aaron Lu | Medium, accessed May 16, 2026,
<https://medium.com/@aaronlu5/i-built-an-ai-log-analyzer-using-mcp-heres-how-it-works-c7ae5d3ec590>